



INSTITUTO POLITÉCNICO NACIONAL



INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

SISTEMAS EN CHIP

Reporte de la práctica 13

“Convertidor Analógico Digital”

Grupo: 6CM1

Integrantes

- García Escamilla Bryan Alexis
- Meléndez Macedonio Rodrigo

Profesor

Aguilar Sánchez Fernando

Fecha de entrega: 06 de marzo de 2026

INTRODUCCIÓN

Convertidor Analógico-Digital

Un Convertidor Analógico-Digital (ADC) es un dispositivo electrónico que actúa como un puente entre el mundo real (que es esencialmente analógico y continuo) y el mundo de los procesadores (que es digital y discreto).

Su función es transformar una magnitud física (voltaje, corriente, temperatura) en un número binario que un sistema digital pueda procesar.

Proceso de conversión

Para transformar una señal continua en datos digitales, el ADC sigue tres pasos fundamentales:

1. **Muestreo (Sampling):** El ADC toma "fotografías" de la señal analógica a intervalos de tiempo constantes. La velocidad a la que hace esto se llama Frecuencia de Muestreo.
2. **Cuantificación:** A cada muestra se le asigna un valor dentro de un rango determinado. Aquí es donde la señal deja de ser infinita y se ajusta a niveles definidos.
3. **Codificación:** El nivel cuantificado se convierte en un código binario (**0110, 1010, etc.**).

Conceptos fundamentales

Dos parámetros definen la calidad y precisión de un ADC:

- **Resolución (Bits):** Determina cuántos "escalones" tiene el convertidor para representar la señal.
 - Un ADC de 8 bits tiene $2^8 = 256$ niveles.
 - Un ADC de 10 bits tiene $2^{10} = 1024$ niveles.
 - A mayor cantidad de bits, más pequeña es la variación de voltaje que el sistema puede detectar.
- **Voltaje de Referencia (V_{ref})** Es el voltaje máximo que el ADC puede medir. Si V_{ref} es 5V y usamos 10 bits, cada unidad digital representa aproximadamente 4.88mV ($5V/1024$).

Arquitecturas Principales

Existen diferentes formas de construir un ADC, dependiendo de si se necesita velocidad o precisión:

- **ADC de Aproximaciones Sucesivas (SAR):** Es el más común en microcontroladores. Utiliza un comparador para ir probando bit por bit (del más significativo al menos significativo) hasta encontrar el valor más cercano al voltaje de entrada. Es equilibrado en velocidad y costo.
- **ADC Flash (Paralelo):** Es extremadamente rápido. Utiliza un comparador para cada nivel de voltaje posible. Se usa en aplicaciones de video u osciloscopios de alta velocidad, pero es muy costoso y consume mucha energía.
- **ADC Delta-Sigma:** Es más lento, pero ofrece una resolución altísima (hasta 24 bits). Se usa en audio de alta fidelidad y básculas de precisión.

El Teorema de Nyquist

En la teoría de conversión, existe una regla de oro: para poder reconstruir una señal analógica a partir de sus muestras digitales, la frecuencia de muestreo debe ser al menos el doble de la frecuencia más alta de la señal original. Si no se cumple, aparece un error llamado Aliasing (falsos componentes de frecuencia).

DESARROLLO

Circuito en hecho en Proteus

Para el desarrollo de la práctica 13, el circuito a armar es el siguiente:

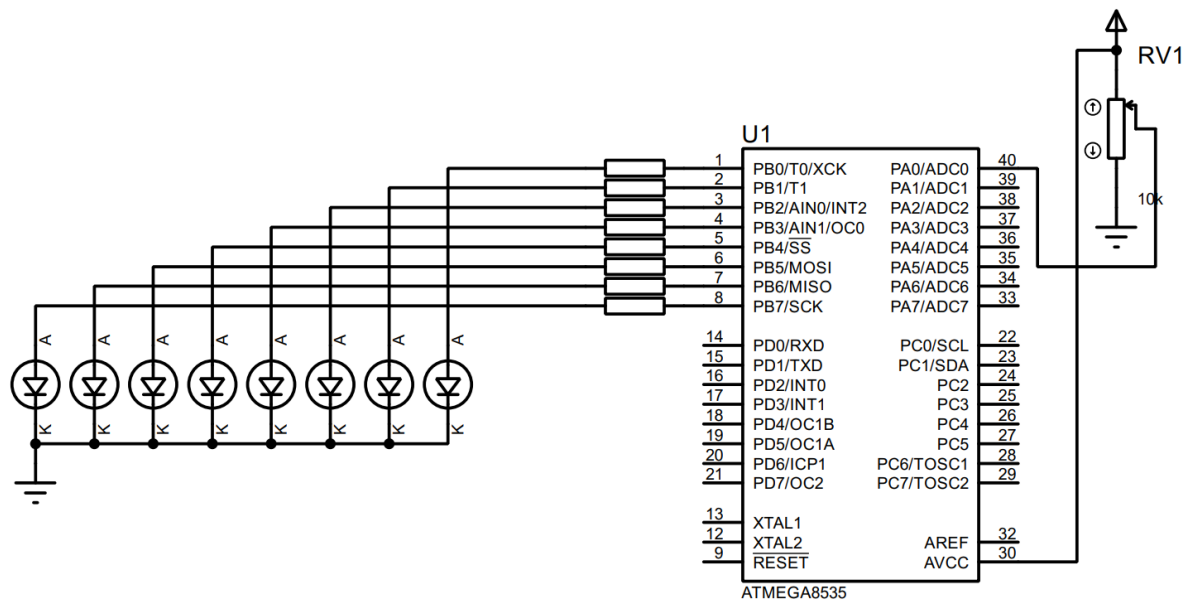


Imagen 1. Circuito creado en Proteus.

Configuración previa al código

De acuerdo con la imagen del circuito, el registro B se configura como de salida (PB0-PB7).

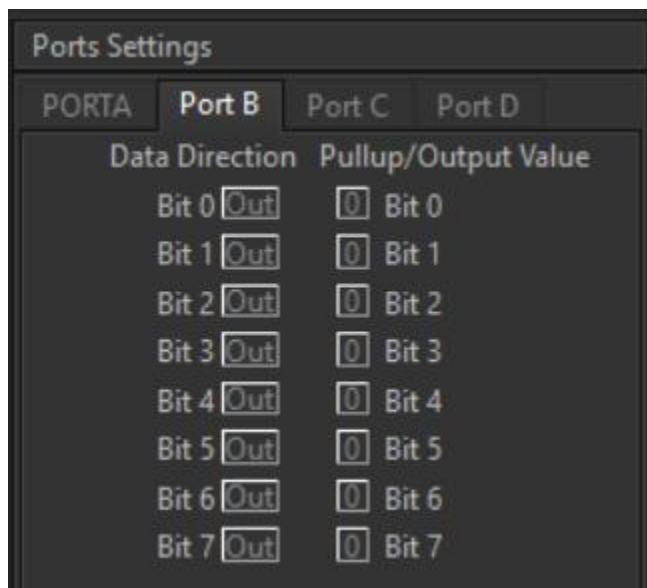


Imagen 2. Configuración del puerto B.

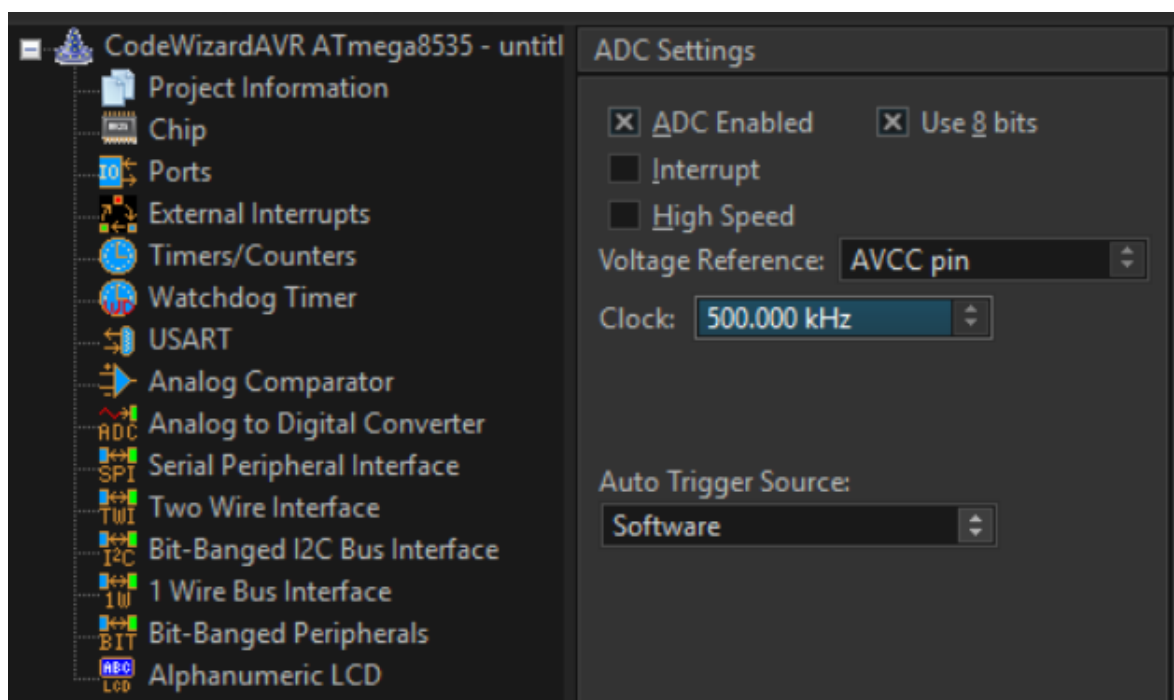


Imagen 3. Activación y configuración del convertidor analógico digital desde CodeVision.

Código del circuito de CodeVision:

```
1  /*****
2  This program was created by the CodeWizardAVR V4.06a
3  Automatic Program Generator
4      Copyright 1998-2025 Pavel Haiduc, HP InfoTech S.R.L.
5  http://www.hpinfotech.ro
6
7  Project : Pr  ctica 13 'Convertidor anal  gico digital'
8  Version : 1.3
9  Date   : 06/03/2026
10 Author : Garc  a Escamilla Bryan Alexis
11 Company :
12 Comments:
13
14
15 Chip type           : ATmega8535
16 Program type        : Application
17 AVR Core Clock frequency: 1.000000 MHz
18 Memory model        : Small
19 External RAM size    : 0
20 Data Stack size     : 128
21 *****/
22
23 // I/O Registers definitions
24 #include <mega8535.h>
25
26 // Delay functions
27 #include <delay.h>
28
29 // Declare your global variables here
30
31 // ADC Voltage Reference: AVCC pin
32 #define ADC_VREF_TYPE ((0<<REFS1) | (1<<REFS0) | (1<<ADLAR))
```

```
35 // of the AD conversion result
36 // Read Voltage=read_adc*(Vref/256.0)
37 unsigned char read_adc(unsigned char adc_input) {
38     ADMUX=adc_input | ADC_VREF_TYPE;
39     // Delay needed for the stabilization of the ADC input voltage
40     delay_us(10);
41     // Start the AD conversion
42     ADCSRA|=(1<<ADSC);
43     // Wait for the AD conversion to complete
44     while ((ADCSRA & (1<<ADIF))==0);
45     ADCSRA|=(1<<ADIF);
46     return ADCH;
47 }
```

```

49 void main(void) {
50     // Declare your local variables here
51
52     // Input/Output Ports initialization
53     // Port A initialization
54     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
55     DDRA=(0<<DDA7) | (0<<DDA6) | (0<<DDA5) | (0<<DDA4) | (0<<DDA3) | (0<<DDA2) | (0<<DDA1) | (0<<DDA0);
56     // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
57     PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) | (0<<PORTA2) | (0<<PORTA1) | (0<<PORTA0);
58
59     // Port B initialization
60     // Function: Bit7=Out Bit6=Out Bit5=Out Bit4=Out Bit3=Out Bit2=Out Bit1=Out Bit0=Out
61     DDRB=(1<<ddb7) | (1<<ddb6) | (1<<ddb5) | (1<<ddb4) | (1<<ddb3) | (1<<ddb2) | (1<<ddb1) | (1<<ddb0);
62     // State: Bit7=0 Bit6=0 Bit5=0 Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=0
63     PORTB=(0<<PORTB7) | (0<<PORTB6) | (0<<PORTB5) | (0<<PORTB4) | (0<<PORTB3) | (0<<PORTB2) | (0<<PORTB1) | (0<<PORTB0);
64
65     // Port C initialization
66     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
67     DDRC=(0<<DDC7) | (0<<DDC6) | (0<<DDC5) | (0<<DDC4) | (0<<DDC3) | (0<<DDC2) | (0<<DDC1) | (0<<DDC0);
68     // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
69     PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5) | (0<<PORTC4) | (0<<PORTC3) | (0<<PORTC2) | (0<<PORTC1) | (0<<PORTC0);
70
71     // Port D initialization
72     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
73     DDRD=(0<<DDD7) | (0<<DDD6) | (0<<DDD5) | (0<<DDD4) | (0<<DDD3) | (0<<DDD2) | (0<<DDD1) | (0<<DDD0);
74     // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
75     PORTD=(0<<PORTD7) | (0<<PORTD6) | (0<<PORTD5) | (0<<PORTD4) | (0<<PORTD3) | (0<<PORTD2) | (0<<PORTD1) | (0<<PORTD0);

```

```

77     // Timer/Counter 0 initialization
78     // Clock source: System Clock
79     // Clock value: Timer 0 Stopped
80     // Mode: Normal top=0xFF
81     // OC0 output: Disconnected
82     TCCR0=(0<<WGM00) | (0<<COM01) | (0<<COM00) | (0<<WGM01) | (0<<CS02) | (0<<CS01) | (0<<CS00);
83     TCNT0=0x00;
84     OCR0=0x00;
85
86     // Timer/Counter 1 initialization
87     // Clock source: System Clock
88     // Clock value: Timer1 Stopped
89     // Mode: Normal top=0xFFFF
90     // OC1A output: Disconnected
91     // OC1B output: Disconnected
92     // Noise Canceler: Off
93     // Input Capture on Falling Edge
94     // Timer1 Overflow Interrupt: Off
95     // Input Capture Interrupt: Off
96     // Compare A Match Interrupt: Off
97     // Compare B Match Interrupt: Off
98     TCCR1A=(0<<COM1A1) | (0<<COM1A0) | (0<<COM1B1) | (0<<COM1B0) | (0<<WGM11) | (0<<WGM10);
99     TCCR1B=(0<<ICNC1) | (0<<ICES1) | (0<<WGM13) | (0<<WGM12) | (0<<CS12) | (0<<CS11) | (0<<CS10);
100    TCNT1H=0x00;
101    TCNT1L=0x00;
102    ICR1H=0x00;
103    ICR1L=0x00;
104    OCR1AH=0x00;
105    OCR1AL=0x00;
106    OCR1BH=0x00;
107    OCR1BL=0x00;

```

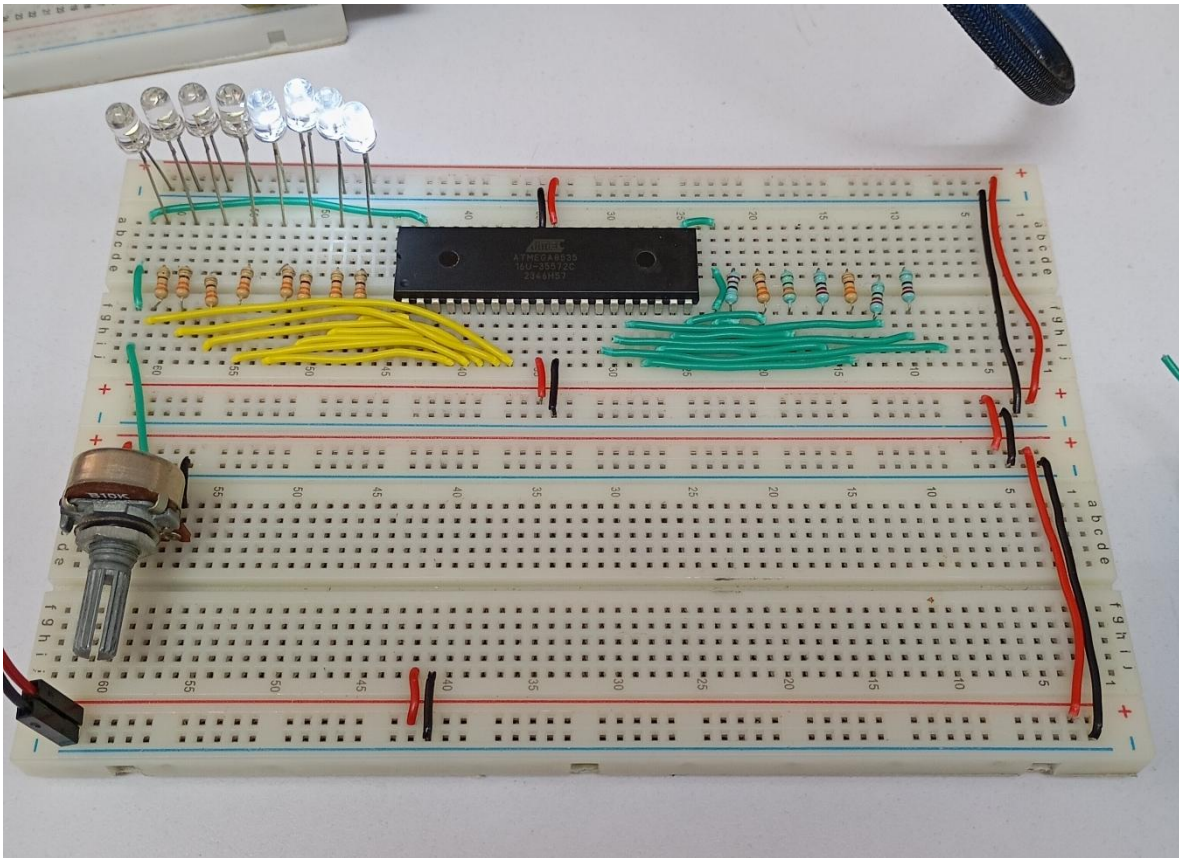


```

109 // Timer/Counter 2 initialization
110 // Clock source: System Clock
111 // Clock value: Timer2 Stopped
112 // Mode: Normal top=0xFF
113 // OC2 output: Disconnected
114 ASSR=(0<<AS2);
115 TCCR2=(0<<WGM20) | (0<<COM21) | (0<<COM20) | (0<<WGM21) | (0<<CS22) | (0<<CS21) | (0<<CS20);
116 TCNT2=0x00;
117 OCR2=0x00;
118
119 // Timer(s)/Counter(s) Interrupt(s) initialization
120 TIMSK=(0<<OCIE2) | (0<<TOIE2) | (0<<TICIE1) | (0<<OCIE1A) | (0<<OCIE1B) | (0<<TOIE1) | (0<<OCIE0) | (0<<TOIE0);
121
122 // External Interrupt(s) initialization
123 // INT0: Off
124 // INT1: Off
125 // INT2: Off
126 MCUCR=(0<<ISC11) | (0<<ISC10) | (0<<ISC01) | (0<<ISC00);
127 MCUCSR=(0<<ISC2);
128
129 // USART initialization
130 // USART disabled
131 UCSRB=(0<<RXCIE) | (0<<TXCIE) | (0<<UDRIE) | (0<<RXEN) | (0<<TXEN) | (0<<UCSZ2) | (0<<RXB8) | (0<<TXB8);
132
133 // Analog Comparator initialization
134 // Analog Comparator: Off
135 // The Analog Comparator's positive input is
136 // connected to the AIN0 pin
137 // The Analog Comparator's negative input is
138 // connected to the AIN1 pin
139 ACSR=(1<<ACD) | (0<<ACBG) | (0<<ACO) | (0<<ACI) | (0<<ACIE) | (0<<ACIC) | (0<<ACIS1) | (0<<ACIS0);
140
141 // ADC initialization
142 // ADC Clock frequency: 500.000 kHz
143 // ADC High Speed Mode: Off
144 // ADC Auto Trigger Source: Software
145 // Only the 8 most significant bits of
146 // the AD conversion result are used
147 ADCSRA=(1<<ADEN) | (0<<ADSC) | (0<<ADATE) | (0<<ADIF) | (0<<ADIE) | (0<<ADPS2) | (0<<ADPS1) | (1<<ADPS0);
148 SFIOR=(1<<ADHSM) | (0<<ADTS2) | (0<<ADTS1) | (0<<ADTS0);
149
150 // SPI initialization
151 // SPI disabled
152 SPCR=(0<<SPIE) | (0<<SPE) | (0<<DORD) | (0<<MSTR) | (0<<CPOL) | (0<<CPHA) | (0<<SPR1) | (0<<SPR0);
153
154 // TWI initialization
155 // TWI disabled
156 TWCR=(0<<TWEA) | (0<<TWSTA) | (0<<TWSTO) | (0<<TWEN) | (0<<TWIE);
157
158 while (1) {
159     // Place your code here
160     PORTB = read_adc(1);
161 }
162 }

```

Circuito físico



CONCLUSIÓN

- **García Escamilla Bryan Alexis**

El desarrollo de esta práctica resultó ser muy sencillo puesto que CodeVision hace todo el trabajo duro, es decir solo hay que configurar el convertidor antes de empezar a escribir código y CodeVision agregará la función `read_adc` al código para que solo sea llamada indicando como parámetro el pin del puerto A de donde se obtendrá el valor analógico, y entonces esta devolverá su correspondiente a 8 bits.

Para ello nos tenemos que dirigir a la sección 'Analog to Digital Converter' en la configuración del proyecto, seleccionar la casilla 'ADC Enabled' que mostrará más opciones de configuración. En este caso se seleccionó la casilla 'Use 8 bits' para representar el valor analógico en 8 bits, ya que por defecto es de 10 bits, pero como se usó el puerto B por eso el cambio.

Con esta configuración la relación de voltaje es la siguiente:

$$Valor\ digital = \frac{Valor\ analógico \times 256}{V_{ref}}$$

Donde en este caso $V_{ref} = 5$.

- **Meléndez Macedonio Rodrigo**

Para esta práctica se pedía hacer un convertidor analógico digital, lo cual resultó ser más sencillo de lo que pensábamos, ya que CodeVisionAVR ofrece demasiadas facilidades al momento de querer convertir un valor analógico en digital.

De inicio, al momento de crear un proyecto, el software tiene una sección llamada "Analog to Digital Converter" en el que tenemos un combo en el que podemos seleccionar "ADC Enabled" y eso nos mostrara más opciones de configuración, ahí seleccionamos "Use 8 bits" para representar el valor analógico de 8 bits. De esta forma, la configuración está completa y CodeVisionAVR se encargó de generar el código en el cual solo asigna al puerto B la función "read_ADC". Con esto, el programa se encarga de convertir los valores analógicos del potenciómetro en valores digitales de 8 bits asignados al puerto B.

BIBLIOGRAFÍA

(s.f.). *Analog-to-digital converter*. Wikipedia. Recuperado el 7 de marzo de 2026.
https://en.wikipedia.org/wiki/Analog-to-digital_converter

(s.f.). *ADC Fundamentals*. Analog Devices. Recuperado el 7 de marzo de 2026.
<https://www.analog.com/en/resources/glossary/adc.html>